Payment & Installment Database Schema

payTrackEdu backend • Generated from the JPA entity model • October 2026

> **The important idea:** a payment-plan installment is a reusable *template*. An enrollment installment is a student-specific *snapshot* created from that template. They are stored in different tables and are not directly linked.

PK primary key FK foreign key soft delete record is hidden, not physically removed

Core relationship diagram

academic_year

- PK id : UUID
- code, start_date, end_date
- status
- soft delete is_deleted, deleted_at

promotion

- PK id : UUID
- code, name, active
- soft delete is_deleted, deleted_at

student

- PK id : UUID
- first_name, last_name, email
- phones, birthday, parent contact
- soft delete is_deleted, deleted_at

promotion_session

- PK id : UUID
- FK academic_year_id → academic_year
- FK promotion_id → promotion
- code, registration_fee, status
- Unique: (academic_year_id, promotion_id)

payment_plan

- PK id : UUID
- FK promotion_session_id → promotion_session
- code, plan, total_amount
- Unique: (promotion_session_id, plan)
- soft delete is_deleted, deleted_at

enrollment

- PK id : UUID
- FK student_id → student
- FK promotion_session_id → promotion_session
- FK payment_plan_id → payment_plan
- registration_fee_amount, tuition_amount
- discount_amount, status

payment_plan_installment (template)

- PK id : UUID
- FK payment_plan_id → payment_plan
- sequence_number, type
- amount, due_date

enrollment_installment (student snapshot)

- PK id : UUID
- FK enrollment_id → enrollment
- sequence_number, type, label
- expected_amount, due_date, status

payment + payment_allocation

- **payment**: FK enrollment_id → enrollment
- **payment_allocation**: FK payment_id → payment
- FK enrollment_installment_id → enrollment_installment

soft delete is_deleted, deleted_at

Rules: unique sequence; active amounts ≤ plan total

soft delete is_deleted, deleted_at

allocation amount is the amount applied

Every table also inherits audit fields: created_at, created_by, updated_at, updated_by.

Why the old installment remains

Current behavior in the application

1. Create plan
Payment plan and its template installments are created.

→

2. Create enrollment
`generateForEnrollment()` copies the registration fee and plan templates into student installments.

→

3. Delete template
The template has `is_deleted = true`. The copied student installment does not change.

→

4. Update enrollment
Only `enrollment.payment_plan_id` and amounts are changed. Existing enrollment installments stay.

There is no database relationship between these two records

Template record	Created student record	What happens after template deletion
`payment_plan_installment.id` Plan: Monthly Tuition 1: 10,000	`enrollment_installment.id` Student: Ahmed Tuition 1: 10,000	The template is soft-deleted, but Ahmed's separate record is still active and returned by the API.

Important: `EnrollmentInstallment` stores only `enrollment_id`; it does not store `payment_plan_installment_id`. Therefore JPA/database cascade cannot automatically know which student installment should be changed when a template changes.

Where this happens in the code

File / method	Current behavior	Effect
`EnrollmentService.create()`	Creates enrollment, then calls `generateForEnrollment(id)`.	Initial student installments are correctly copied.
`EnrollmentInstallmentService.buildInstallments()`	Reads active plan templates and creates new independent student installments.	It creates a snapshot, not a live link.
`PaymentPlanInstallmentService.delete()`	Sets only the template's `deleted` flag.	Existing enrollments are unaffected.
`EnrollmentService.update()`	Changes enrollment fields but does not cancel, delete, or regenerate enrollment installments.	The old student schedule remains visible.
`generateForEnrollment()`	Refuses to run if non-cancelled student installments already exist.	Simply calling it after an update will result in a conflict.

Implemented safe behavior

> **Rule now applied:** when a payment-plan installment is created, updated, deleted, or restored, the schedules of all affected enrollments with no active payment allocation are rebuilt from the current plan templates. An enrollment that already has allocated payments is never changed automatically.

Safe update workflow

1. Load the enrollment and detect whether `request.paymentPlanId()` differs from the current payment plan ID. If it is the same plan, do not rebuild the schedule.
2. Load all existing `enrollment_installment` rows with a lock. Check whether any has a `payment_allocation` (or a paid amount above zero).
3. If payments exist, reject the plan change with HTTP 409 and a clear message. This protects financial history and avoids orphaning payment allocations.
4. If there are no payments, soft-delete or cancel all current enrollment installments. Do this in the same transaction as the enrollment update.
5. Save the new payment plan and amounts on the enrollment, then generate a new set of enrollment installments from the current active templates.
6. Return the enrollment and retrieve its installments again. The API should now return only the new active schedule.

Data integrity rules to keep

Rule	Reason
Do not delete an enrollment installment physically when it has payment allocations.	`payment_allocation.enrollment_installment_id` depends on it; payment history must remain auditable.
Use soft delete (`is_deleted = true`) for unpaid old installments.	The application already uses soft deletion and its queries filter on `deleted = false`.
Regenerate only after old rows are made inactive.	`generateForEnrollment()` deliberately prevents duplicate active schedules.
Do not rebuild an enrollment with active payment allocations.	Its schedule is financial history. A template change still applies to unpaid enrollments and future enrollments.
The sum of active plan installments cannot exceed `payment_plan.total_amount`.	A fourth 3,000 installment is rejected when the plan total is 9,000. The plan total also cannot be lowered below its current active installment total.

Implemented service methods and endpoint

`EnrollmentInstallmentService.synchronizeEnrollmentInstallments(enrollmentId)` replaces one unpaid schedule. `synchronizeUnpaidEnrollmentsForPaymentPlan(paymentPlanId)` is invoked after every plan-installment change. Updating an enrollment to a different plan also synchronizes its schedule.

The frontend can manually request a refresh with `POST /api/enrollment-installments/enrollment/{enrollmentId}/synchronize`. It returns HTTP 409 if the enrollment has active payment allocations.

This change uses the existing schema; it adds service logic only.